



VIT[®]
B H O P A L

www.vitbhopal.ac.in

PROJECT REPORT

Vityarthi AI/ML Project

**An AI-based Intelligent Route Optimization System
using Python and Prolog**

*A hybrid AI approach for finding efficient routes based on map data,
constraints, and user requirements.*

Submitted for

VITYarthi – Build Your Own Project

VIT Bhopal University

Submitted by

AYAN SHARMA

Reg. No.: 25MIM10136

Programme: Integrated M.Tech (Artificial Intelligence)

VIT Bhopal University

September 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and Target Users
5. Functional Requirements
6. Non-Functional Requirements
7. Technologies Used
8. System Architecture
9. Design Diagrams
10. Campus Map and Knowledge Representation
11. Route Search Method
12. Dynamic Constraint Handling
13. Python–Prolog Integration
14. Command-Line Execution
15. Design Decisions and Rationale
16. Implementation Details
17. Testing Approach and Results
18. Challenges Faced
19. Advantages
20. Limitations
21. Learnings and Key Takeaways
22. Future Enhancements
23. Conclusion
24. References
25. Project Repository
26. Developer

1. Introduction

The **Intelligent Route Optimization System** is a command-line Artificial Intelligence project that demonstrates knowledge representation, logical reasoning, recursive search, and constraint handling using Python and Prolog.

The system models a simulated university-campus route network containing locations such as the Main Gate, Administration Block, Academic Block, Central Library, Research Block, Auditorium, Cafeteria, Computer Science Block, Medical Center, Sports Complex, Student Activity Center, Hostel Block, Residential Area, Parking Lot, and Central Plaza.

Python provides the command-line interface and application control, while Prolog stores the route knowledge and performs logical path reasoning. Locations can be marked as blocked or restricted before route search.

Scope note: The campus map and route distances are simulated for academic demonstration and are not intended to represent an official VIT Bhopal navigation service.

2. Problem Statement

Finding an efficient route becomes more challenging when one or more locations are temporarily unavailable. The system therefore represents connections and distances, searches possible paths, avoids unavailable locations, compares valid routes, and reports when no valid route exists.

3. Objectives

- Implement a route-planning problem using Artificial Intelligence concepts.
- Represent the simulated environment using Prolog facts and rules.
- Use recursive search-based reasoning to find valid routes.
- Calculate total route distance and select the shortest valid route.
- Integrate Python and Prolog using PySwip.
- Handle dynamic blocked and restricted locations.
- Provide clear command-line results.

4. Scope and Target Users

Scope: Predefined simulated campus network; no GPS, live traffic, or external mapping service.

Target users: AI/logic-programming students, faculty evaluators, and developers exploring Python–Prolog integration.

5. Functional Requirements

- **FR1.** Accept start and destination locations.
- **FR2.** Load route facts and rules.
- **FR3.** Apply blocked and restricted locations.
- **FR4.** Search for valid paths.
- **FR5.** Calculate route distance.
- **FR6.** Select the shortest valid route.
- **FR7.** Display the route and distance.
- **FR8.** Report when no valid route exists.

6. Non-Functional Requirements

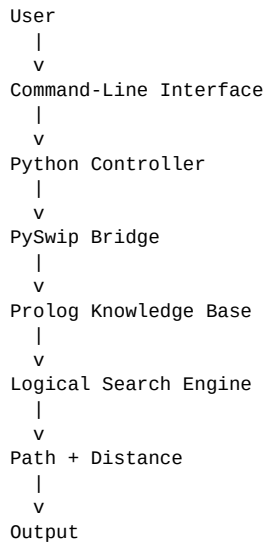
- **NFR1.** Usability — simple terminal commands and understandable output.

- **NFR2.** Reliability — unavailable locations are excluded from route calculation.
- **NFR3.** Maintainability — Python and Prolog responsibilities are separated.
- **NFR4.** Performance — efficient operation for the simulated network.
- **NFR5.** Error handling — clear reporting when route calculation fails.
- **NFR6.** Portability — uses Python, SWI-Prolog, and PySwip.

7. Technologies Used

Python; Prolog; SWI-Prolog; PySwip 0.3.3; Python argparse; Windows Command Prompt/PowerShell; GitHub.

8. System Architecture



9. Design Diagrams

Workflow: User Input → Validate Request → Load Knowledge Base → Apply Constraints → Search Valid Paths → Select Minimum Distance → Display Result.

Use Cases: Enter route request; set blocked/restricted locations; view route and distance; view no-route result.

Sequence: User request → Python loads Prolog → constraints cleared → constraints asserted → `shortest_path` query → Prolog search → result returned → Python output.

Component view: Command-Line User → `route_optimizer.py` → PySwip → `knowledge_base.pl` → Path + Distance.

10. Campus Map and Knowledge Representation

The project uses the following simulated campus map as its conceptual environment.

<reportlab.lib.utils.ImageReader object at 0x7f372d94ba20>

Figure 1. Simulated campus environment used for the academic project.

The network contains 15 locations: Main Gate, Administration Block, Academic Block, Central Library, Research Block, Auditorium, Cafeteria, Computer Science Block, Medical Center, Sports Complex, Student Activity Center, Hostel Block, Residential Area, Parking Lot, and Central Plaza.

A route is represented as `connected('Main Gate', 'Central Library', 250)`. The project treats the network as bidirectional through Prolog route rules.

11. Route Search Method

The Prolog search recursively explores legal moves, tracks visited locations to prevent cycles, accumulates route distance, and compares valid candidate paths. The `shortest_path` predicate returns the candidate with minimum total distance.

12. Dynamic Constraint Handling

Blocked location: A location can be excluded using the `--blocked` command-line option.

```
python route_optimizer.py --start "Main Gate" --end "Central Library" --blocked "Administration Block"
```

Restricted location: A location can be excluded using the `--restricted` option.

```
python route_optimizer.py --start "Main Gate" --end "Central Library" --restricted "Central Plaza"
```

13. Python–Prolog Integration

PySwip connects Python with SWI-Prolog. Python loads the knowledge base, clears previous temporary constraints, adds current constraints, and executes `shortest_path(Start, Destination, Path, Distance)`.

14. Command-Line Execution

```
python route_optimizer.py --start "Main Gate" --end "Central Library"  
python route_optimizer.py --start "Main Gate" --end "Central Library" --blocked "Central Plaza"  
python route_optimizer.py --start "Main Gate" --end "Central Library" --restricted "Administration Block"
```

15. Design Decisions and Rationale

- **Prolog:** Suitable for facts, rules, constraints, and symbolic search.

- **Python:** Provides command-line control and readable output formatting.
- **PySwip:** Provides Python–SWI-Prolog integration.
- **Dynamic constraints:** Allow route availability to change between executions.
- **Recursive search:** Explores connected locations while tracking visited nodes.
- **Distance-based selection:** Compares accumulated distances to select the shortest valid route.

16. Implementation Details

route_optimizer.py: command-line arguments, Prolog initialization, constraint management, route query, and output.

knowledge_base.pl: route facts, bidirectional rules, dynamic constraints, availability rules, recursive search, and shortest-path selection.

requirements.txt: PySwip==0.3.3.

README.md: setup, execution, testing, architecture, and project structure.

statement.md: problem statement, scope, target users, features, and expected input/output.

17. Testing Approach and Results

Testing focuses on normal route calculation and dynamic constraint handling.

Test	Condition	Expected	Status
TC-01	Normal Main Gate → Central Library	Valid route and distance	PASS
TC-02	Central Library blocked	No valid route	PASS
TC-03	Central Library restricted	No valid route	PASS
TC-04	Administration Block restricted	Alternative valid route considered	PASS
TC-05	Multiple intermediate locations blocked	Unavailable locations avoided	PASS

18. Challenges Faced

- Python–SWI-Prolog integration.
- Runtime constraint handling.
- Cycle prevention during recursive search.
- No-route handling.
- Maintaining command-line execution.

19. Advantages

- Demonstrates symbolic AI.
- Uses logical knowledge representation.
- Supports dynamic constraints.
- Provides route distance.
- Separates application logic from logical knowledge.

20. Limitations

- Simulated predefined network.
- Manually defined distances.
- Command-line interface.
- No live GPS or traffic data.
- Intended for academic demonstration.

21. Learnings and Key Takeaways

- Prolog facts and rules can model practical environments.
- Recursive search can solve pathfinding problems.
- Runtime constraints can modify route availability.
- PySwip enables Python–Prolog integration.
- Testing is essential for successful and no-solution cases.

22. Future Enhancements

- Graphical route visualization.
- Interactive map display.
- Larger route networks.
- Real-time location and traffic information.
- Multiple optimization criteria.
- Automated tests and richer logging.
- Real-world map-service integration.

23. Conclusion

The Intelligent Route Optimization System demonstrates knowledge representation, logical reasoning, recursive search, dynamic constraints, and distance-based route selection using Python and Prolog. Testing demonstrates route discovery under normal conditions and rejection of unavailable destinations.

24. References

1. SWI-Prolog documentation.
2. PySwip documentation.
3. Python documentation.
4. VITyarthi Build Your Own Project guidelines.

25. Project Repository

<https://github.com/AyanSharma01/Vityarthi-AI-ML-Project>

26. Developer

Ayan Sharma
Integrated M.Tech in Artificial Intelligence
VIT Bhopal University
Registration Number: 25MIM10136